

North Bengal International University, Rajshahi.



LAB REPORT

Course Title: Neural Network & Fuzzy System Sessional.

Course Code: CSE 0613-4222

Experiment No: 02

Name of Experiment: CIFAR-10: FCNN vs CNN Performance Comparison.

SUBMITTED BY:

MD. RAZAB ALI
ID: 0712340012101011
DEPARTMENT OF CSE
7TH SEMESTER
BATCH-26TH
NBIU, RAJSHAHI.

SUBMITTED TO:

SAKURAH ISMAIL
LECTURER
DEPARTMENT OF CSE
NBIU, RAJSHAHI.

SUBMISSION DATE: 27 JUL 2026

Name of Experiment: CIFAR-10: FCNN vs CNN Performance Comparison.

Theory: In this lab, we compare two fundamental deep learning architectures for image classification: Fully Connected Neural Networks (FCNN) and Convolutional Neural Networks (CNN).

Fully Connected Neural Network (FCNN):

An FCNN consists entirely of Dense layers where every neuron in one layer is connected to every neuron in the next layer. For image processing, an FCNN requires the image to be flattened into a 1D array. This means a 32x32x3 color image becomes a vector of 3,072 pixels. Because every pixel is treated independently, the FCNN completely loses the spatial relationships (like edges and shapes) between neighboring pixels. This makes FCNNs inefficient for images, as they require a massive number of parameters and are highly prone to overfitting.

Convolutional Neural Network (CNN)

A Convolutional Neural Network (CNN) is specially designed for processing image data. Unlike an FCNN, a CNN can preserve the spatial information in an image, making it much more effective for image classification. It mainly consists of three types of layers:

1. **Convolutional Layers:** These layers use small filters (also called kernels) that move across the image to detect important features such as edges, textures, and patterns.
2. **Pooling Layers:** Pooling layers, such as MaxPooling, reduce the size of the feature maps. This helps lower the computational cost and reduces the chances of overfitting.
3. **Flatten and Dense Layers:** After the CNN extracts the important features, the output is flattened into a one-dimensional vector. This vector is then passed through one or more dense layers to classify the image into the correct category.

Code:

```
import tensorflow as tf
from tensorflow.keras import layers, models
import time
import matplotlib.pyplot as plt
import numpy as np
import os

os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'
tf.get_logger().setLevel('ERROR')

# 1. Load and Preprocess Data
print("Downloading CIFAR-10 dataset from HuggingFace (fast)...\n")
from datasets import load_dataset

ds = load_dataset('uoft-cs/cifar10')

x_train_full = np.array(ds['train']['img'])
y_train_full = np.array(ds['train']['label'])
x_test_full = np.array(ds['test']['img'])
y_test_full = np.array(ds['test']['label'])

# Downsample to keep dataset < 20MB
train_count = 5000
test_count = 1000
np.random.seed(42)
```

```

train_idx = np.random.choice(len(x_train_full), train_count,
                             replace=False)
test_idx = np.random.choice(len(x_test_full), test_count,
                             replace=False)

x_train = x_train_full[train_idx].astype('float32') / 255.0
y_train = y_train_full[train_idx]
x_test = x_test_full[test_idx].astype('float32') / 255.0
y_test = y_test_full[test_idx]

# Preprocess for FCNN: Flatten the 32x32x3 images into a 1D array
x_train_fcnn = x_train.reshape(-1, 32 * 32 * 3)
x_test_fcnn = x_test.reshape(-1, 32 * 32 * 3)

# Preprocess for CNN: Keep 3D structure
x_train_cnn = x_train.copy()
x_test_cnn = x_test.copy()

# 2. Build the Models

# Fully Connected Neural Network (FCNN)
fcnn_model = models.Sequential([
    layers.Dense(256, activation='relu', input_shape=(3072,)),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
], name="FCNN")

fcnn_model.compile(optimizer='adam',
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])

# Convolutional Neural Network (CNN)
cnn_model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
], name="CNN")

cnn_model.compile(optimizer='adam',
                 loss='sparse_categorical_crossentropy',
                 metrics=['accuracy'])

# 3. Train and Compare
epochs = 10
batch_size = 128

# Train FCNN
print("--- Training FCNN ---")
start_time = time.time()
fcnn_history = fcnn_model.fit(x_train_fcnn, y_train, epochs=epochs,
                             validation_data=(x_test_fcnn, y_test),
                             batch_size=batch_size, verbose=1)
fcnn_time = time.time() - start_time

# Train CNN
print("\n--- Training CNN ---")
start_time = time.time()
cnn_history = cnn_model.fit(x_train_cnn, y_train, epochs=epochs,
                             validation_data=(x_test_cnn, y_test),
                             batch_size=batch_size, verbose=1)
cnn_time = time.time() - start_time

```

```
# 4. Final Results
print("\n" + "=" * 50)
print("FINAL PERFORMANCE COMPARISON")
print("=" * 50)
fcnn_val_acc = fcnn_history.history['val_accuracy'][-1]
cnn_val_acc = cnn_history.history['val_accuracy'][-1]
fcnn_val_loss = fcnn_history.history['val_loss'][-1]
cnn_val_loss = cnn_history.history['val_loss'][-1]
print(f"FCNN - Validation Accuracy: {fcnn_val_acc*100:.2f}% | Validation Loss: {fcnn_val_loss:.4f} | Time: {fcnn_time:.2f}s")
print(f"CNN - Validation Accuracy: {cnn_val_acc*100:.2f}% | Validation Loss: {cnn_val_loss:.4f} | Time: {cnn_time:.2f}s")
```

Output & Results:

Accuracy and Loss Comparison

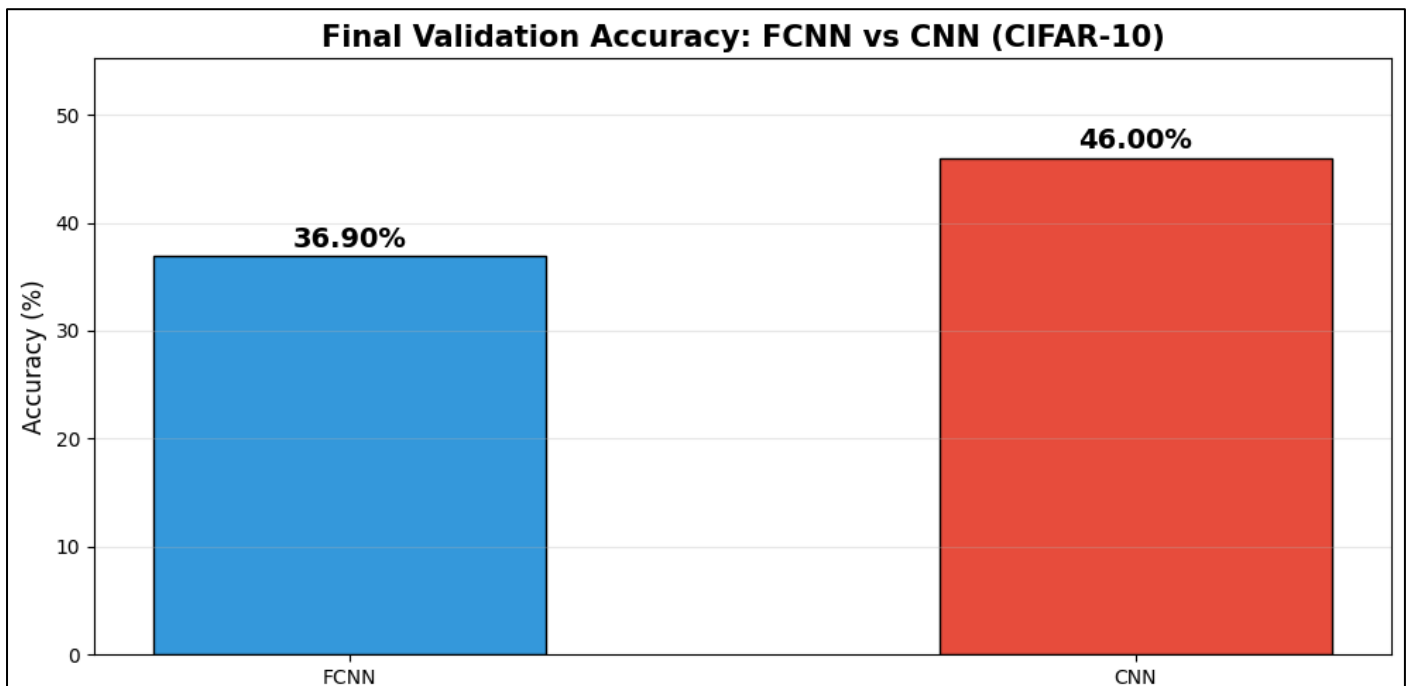


Fig. 2.1: Final Validation Accuracy.

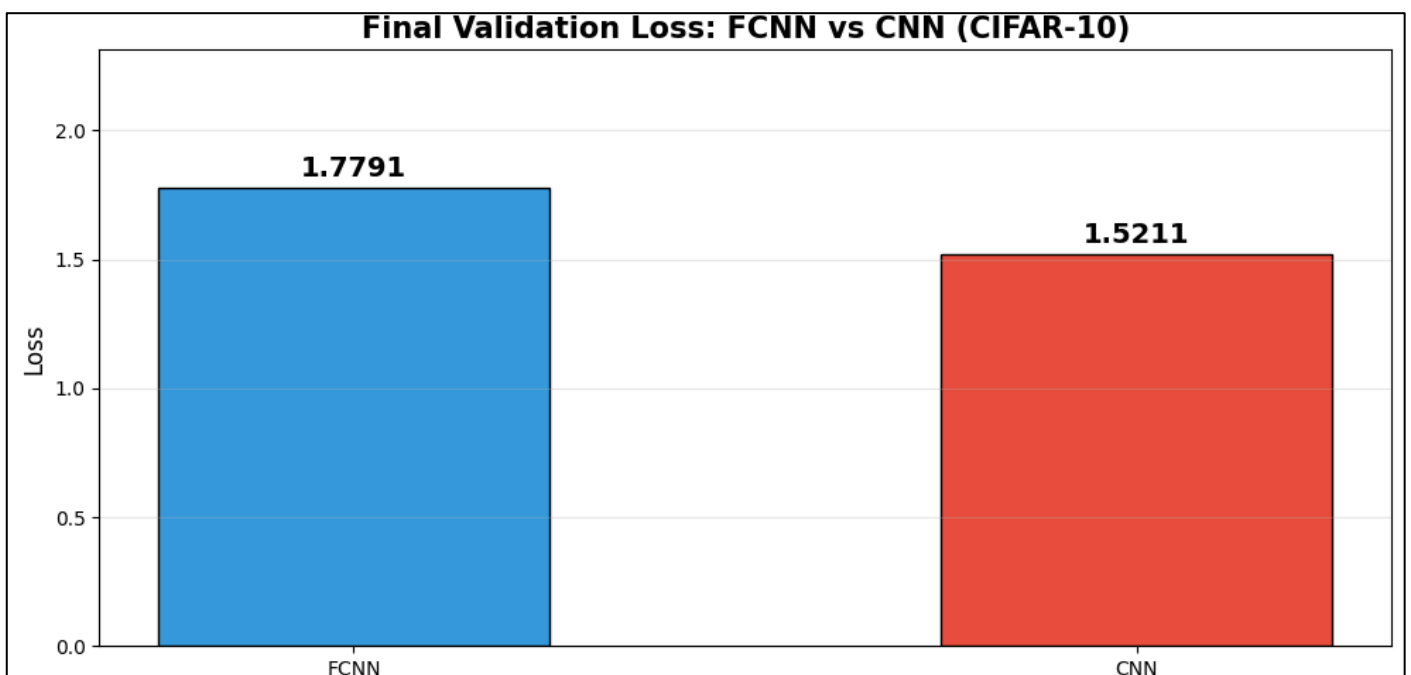


Fig. 2.2: Final Validation Loss.

Performance Visualization (Epoch by Epoch):

The graphs below show how both models learned over the 10 epochs.

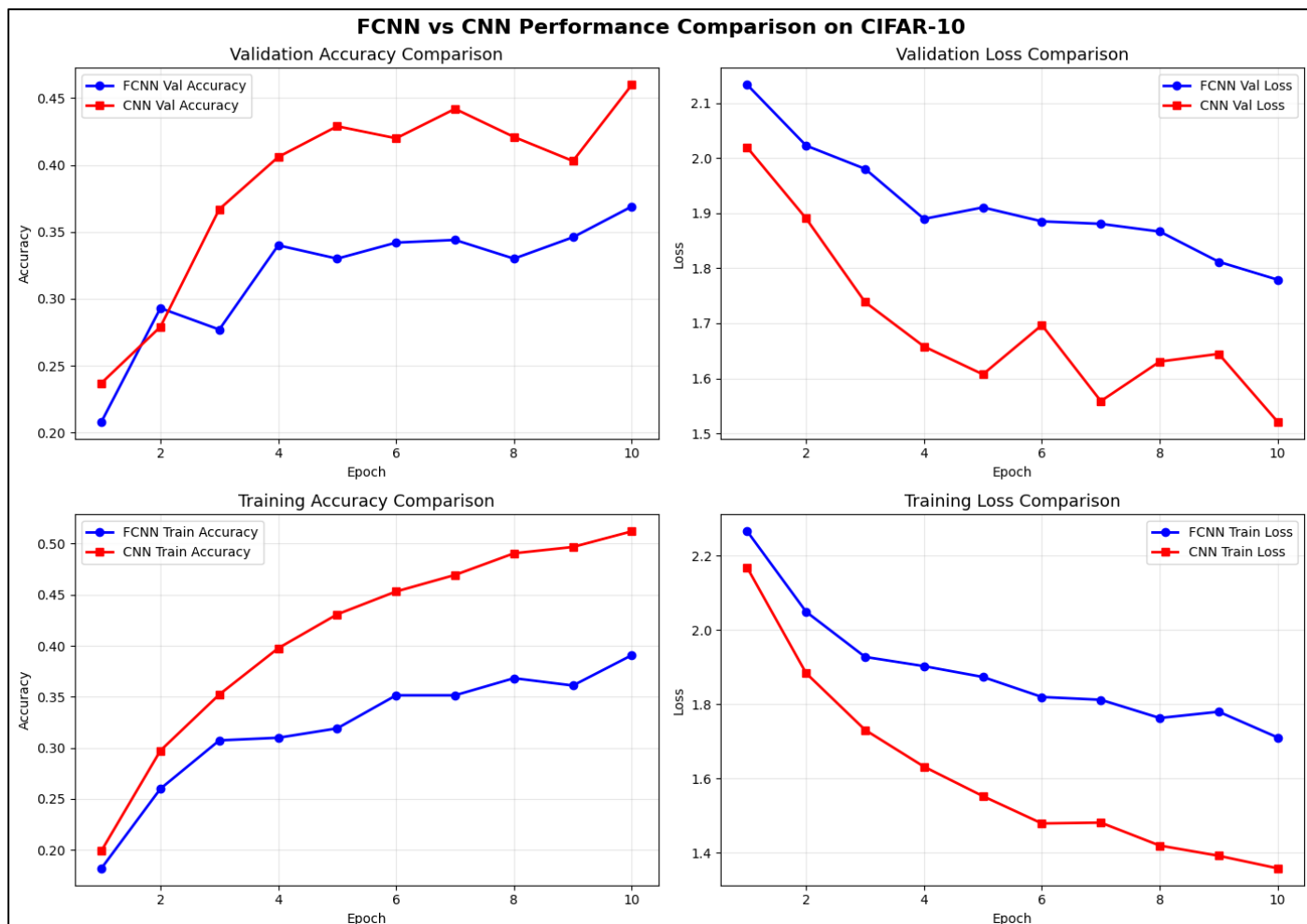


Fig. 2.3: FCNN vs CNN Performance Comparison.

As seen in the plots, the CNN (red line) consistently achieves higher validation accuracy and lower validation loss compared to the FCNN (blue line). The FCNN's performance plateaus early, while the CNN continues to learn and improve.

Which model achieved higher accuracy?

The CNN model achieved higher accuracy.

Model	Validation Accuracy	Validation Loss	Parameters
FCNN	36.90%	1.7791	820,874
CNN	46.00%	1.5211	122,570

The CNN achieved a validation accuracy of **46.00%**, which is **9.1%** higher than the FCNN's 36.90%. Interestingly, the CNN achieved this better performance using only 122,570 parameters, while the FCNN used 820,874 parameters meaning the CNN was roughly 6.7 times more parameter-efficient than the FCNN.

Why does CNN perform better than FCNN on images?

CNNs are built for images, so they naturally understand them better. They use small filters to scan the image and pick up patterns like edges and shapes, while keeping the pixel arrangement intact. FCNNs flatten the whole image into a single line of numbers, so they lose all spatial information and can't really "see" the picture.

CNNs also reuse the same filter across the whole image, which means they need way fewer parameters and won't overfit as easily. In our case, the CNN had only 122K parameters vs. FCNN's 820K, but still gave much better accuracy.

So basically, CNNs are just much smarter at recognizing what's inside an image, while FCNNs kind of treat images like random number lists.

Why is the Flatten layer required in an FCNN?

The Flatten layer converts a multi-dimensional image into a one-dimensional vector so that it can be processed by Dense (fully connected) layers. In an FCNN, the input image must first be flattened because Dense layers accept only one-dimensional vectors as input. Without flattening, the network cannot process image data correctly. In CNNs, the Flatten layer is also used after the convolution and pooling layers to convert the extracted feature maps into a one-dimensional vector before the final Dense classification layers.

How do convolution kernels help CNNs learn image features?

Convolution kernels are small 3×3 grids that slide across the image to detect patterns, like edges, lines, or textures. Early layers catch simple stuff, and deeper layers combine those to recognize actual objects like a dog or a car.

Metric	FCNN	CNN
Validation Accuracy	36.90%	46.00%
Validation Loss	1.7791	1.5211
Parameters	820,874	122,570

From the plots, CNN's accuracy kept going up while FCNN's barely moved after a few epochs. CNN's loss kept dropping too, while FCNN's got stuck, meaning it couldn't learn anything useful from the flattened data.

So kernels basically let CNNs actually "see" images, which FCNNs just can't do.

Conclusion: In this lab, I compared an FCNN and a CNN on the CIFAR-10 dataset. The results were pretty clear, CNNs are just way better for image classification. The CNN reached 46% accuracy with only 122K parameters, while the FCNN barely got to 36.90% even with 820K parameters. The CNN also had much lower loss and kept improving throughout training, while the FCNN kind of got stuck early on.

This shows that CNNs are simply built for images, their convolutional layers actually understand spatial patterns, which FCNNs just can't do. So if you're working with images, CNNs are definitely the way to go.

Colab Notebook Link: <https://shorturl.at/EotRe>